

Week 03: Knowledge, Rules and Expert Systems

Artificial Intelligence Applications in Engineering (MUH-920 Mühendislikte Yapay Zeka Uygulamaları). Lecture notes

Prof. Dr. Utku Kose

Full Professor, Department of Computer Engineering, Süleyman Demirel University, Isparta, Türkiye
Founding Director, AI Application and Research Center (YAZEM), Süleyman Demirel University
Head of the Computer Science Division, Department of Computer Engineering, Süleyman Demirel University

Additional affiliations: University of North Dakota (USA), Universidad Panamericana (Mexico City, Mexico), Vel Tech University (Chennai, India)

IEEE Senior Member, ACM Professional Member

ORCID 0000-0002-9652-6415 | utkukose.com | github.com/utkukose

utkukose@sdu.edu.tr | utku.kose@und.edu | ukose@up.edu.mx | utkukose@gmail.com

Week overview, study path and tasks: <https://github.com/utkukose/ai-applications-in-engineering-NB-lecture/blob/main/weeks/week-03/README.md>

Lecture page with animations and knowledge checks: <https://utkukose.github.io/ai-applications-in-engineering-NB-lecture/weeks/week-03/lecture.html>

Interactive lab with self-assessment and reflection: <https://utkukose.github.io/ai-applications-in-engineering-NB-lecture/weeks/week-03/lab.html>

Colab notebook with Python step 3 and the hands-on work:

https://colab.research.google.com/github/utkukose/ai-applications-in-engineering-NB-lecture/blob/main/weeks/week-03/NB03_rule_based_systems.ipynb

Content licensed under CC BY 4.0. This course is updated in line with current developments in the field.

Last update: October 2026.

Overview

Before machine learning dominated the field, artificial intelligence reached industry through expert systems: programs that store the knowledge of specialists as rules and draw conclusions with an inference engine [1, 2]. DENDRAL inferred molecular structures, MYCIN recommended antibiotics and R1 configured computer orders for a manufacturer [3, 4, 5]. Engineering is full of explicit knowledge in the same form: standards, codes, classification systems and diagnostic procedures. This week builds a small rule engine, applies it to the Unified Soil Classification System and to pump diagnosis, and discusses why transparent rules remain valuable next to learned models [6, 7].

Estimated study time: 8 to 10 hours.

Learning outcomes

By the end of the week, students are expected to represent engineering knowledge as facts and IF-THEN rules, to trace forward and backward chaining by hand, to explain how an expert system justifies its conclusions, to combine certainty factors, to encode a standard classification procedure as a rule base, and to name the strengths and limits of knowledge-based systems.

Python in this week

These notes close with Python step 3: Dictionaries and functions. It covers dictionaries, functions with parameters, default values, return values and docstrings, and rules stored as data, applied to soil classification and pump diagnosis [8]. In the course materials, the step is taught at the start of the Colab notebook, where each snippet runs in a cell next to its explanation and short exercises give immediate feedback, and later sections of the notebook mark further constructs as Python moves.

Knowledge-based systems

A knowledge-based system separates what is known from how it is used. The knowledge base holds facts about the current case and general rules of the domain. The inference engine applies the rules to the facts to derive new facts, and an explanation facility reports which rules led to a conclusion [2]. Because the rules are separate from the engine, experts can read, review and extend the knowledge without changing the program.

Three systems defined the approach. DENDRAL, started at Stanford in the 1960s, generated candidate molecular structures from mass spectrometry data and pruned them with chemical rules; its developers later called it the first expert system for scientific hypothesis formation [3]. MYCIN diagnosed bacterial infections and recommended therapy with about 450 rules, and its evaluation showed performance comparable to specialists [4]. R1, later called XCON, configured computer systems for Digital Equipment Corporation and was among the first expert systems used in daily industrial operation [5]. Building such systems revealed the knowledge acquisition bottleneck: Experts find it hard to state their knowledge as complete and consistent rules, and the rule base needs continuous maintenance as products and practices change [1].

Check your understanding. What is the main architectural idea of a knowledge-based system?

- A. Knowledge is learned automatically from data without experts
- B. Domain knowledge is stored separately from the inference engine that applies it
- C. All decisions are made by a neural network
- D. The program is rewritten for every new case

Answer: B. Separating the knowledge base from the inference engine lets experts inspect and extend rules without changing the engine.

Facts, rules and logic

Propositional logic gives the simplest representation. A fact is a statement that is true or false, such as “the fines content is above 50 percent”. A rule states that a conclusion follows when its conditions hold: IF high vibration AND a peak at twice the running speed THEN misalignment is suspected. Rules whose conditions are conjunctions of facts and whose conclusion is a single fact are called Horn clauses, and for them inference is both simple and efficient [9]. The basic inference step is modus ponens: From “A” and “IF A THEN B”, conclude “B”.

Richer representations exist. First-order logic introduces objects, relations and quantifiers, so that one rule can speak about every pump in a plant. Frames and object-oriented representations group attributes of a concept, and decision tables list combinations of conditions with their actions, a format that engineers know from codes and standards. Whatever the representation, the knowledge must be complete for the intended cases, consistent so that rules do not contradict each other, and specific enough that the engine can test it.

Formulas: Propositional logic as arithmetic

A proposition, such as “the vibration is high”, is either true or false. Writing $v(A)$ for the truth value of a proposition A , with $v(A)=1$ for true and $v(A)=0$ for false, the connectives not ($\neg$), and ($\wedge$), or ($\vee$) and implies ($\rightarrow$) can be computed with ordinary arithmetic [9]:

$$v(\neg A) = 1 - v(A)$$

$$v(A \wedge B) = \min(v(A), v(B)), v(A \vee B) = \max(v(A), v(B))$$

$$v(A \rightarrow B) = \max(1 - v(A), v(B))$$

The function min returns the smaller and max the larger of its arguments. The last formula states that the rule “IF A THEN B ” is false only when A is true and B is false, so $A \rightarrow B$ always has the same truth value as $\neg A \vee B$. A rule whose conditions $p_1, p_2, \dots, p_k$ are joined by and,

$$p_1 \wedge p_2 \wedge \dots \wedge p_k \rightarrow q$$

therefore has the same truth value as the clause

$$\neg p_1 \vee \neg p_2 \vee \dots \vee \neg p_k \vee q$$

in which only one part, q , is not negated. A clause with at most one part that is not negated is a Horn clause [9]. Modus ponens states that $v(A)=1$ and $v(A \rightarrow B)=1$ together force $v(B)=1$. Week 4 keeps the min and max formulas but allows truth values anywhere between 0 and 1 [11].

Worked example: Checking modus ponens and a rule with truth tables

Step 1. The truth table of a rule. All four combinations of $v(A)$ and $v(B)$ are evaluated with the formula for the implication:

$v(A)$	$v(B)$	$1 - v(A)$	$v(A \rightarrow B)$
0	0	$1 - 0 = 1$	$\max(1, 0) = 1$
0	1	$1 - 0 = 1$	$\max(1, 1) = 1$
1	0	$1 - 1 = 0$	$\max(0, 0) = 0$
1	1	$1 - 1 = 0$	$\max(0, 1) = 1$

Step 2. Modus ponens. Only the last row has both $v(A)=1$ and $v(A \rightarrow B)=1$, and in this row $v(B)=1$. Whenever a fact and a rule that starts from it are both true, the conclusion of the rule is true as well.

Step 3. A tempting error. The second row has $v(B)=1$ and $v(A \rightarrow B)=1$, but $v(A)=0$. Concluding A from B and $A \rightarrow B$ is therefore not valid. For diagnosis this means that a symptom predicted by a fault does not prove the fault, because the same symptom can have another cause.

Step 4. A rule in Horn form. Rule R2 of the pump rule base reads IF high vibration (V) AND a peak at twice the running speed (P) THEN misalignment (M). Its truth value is $\max(1 - \min(v(V), v(P)), v(M))$ for every combination of the three facts:

$v(V), v(P), v(M)$	Rule R2
0, 0, 0	$\max(1 - \min(0, 0), 0) = \max(1 - 0, 0) = 1$
0, 0, 1	$\max(1 - \min(0, 0), 1) = \max(1 - 0, 1) = 1$
0, 1, 0	$\max(1 - \min(0, 1), 0) = \max(1 - 0, 0) = 1$
0, 1, 1	$\max(1 - \min(0, 1), 1) = \max(1 - 0, 1) = 1$
1, 0, 0	$\max(1 - \min(1, 0), 0) = \max(1 - 0, 0) = 1$
1, 0, 1	$\max(1 - \min(1, 0), 1) = \max(1 - 0, 1) = 1$
1, 1, 0	$\max(1 - \min(1, 1), 0) = \max(1 - 1, 0) = 0$
1, 1, 1	$\max(1 - \min(1, 1), 1) = \max(1 - 1, 1) = 1$

The Horn clause $\neg V \vee \neg P \vee M$ has the truth value $\max(1 - v(V), 1 - v(P), v(M))$:

$v(V), v(P), v(M)$	Horn clause
0, 0, 0	$\max(1 - 0, 1 - 0, 0) = \max(1, 1, 0) = 1$
0, 0, 1	$\max(1 - 0, 1 - 0, 1) = \max(1, 1, 1) = 1$
0, 1, 0	$\max(1 - 0, 1 - 1, 0) = \max(1, 0, 0) = 1$
0, 1, 1	$\max(1 - 0, 1 - 1, 1) = \max(1, 0, 1) = 1$
1, 0, 0	$\max(1 - 1, 1 - 0, 0) = \max(0, 1, 0) = 1$
1, 0, 1	$\max(1 - 1, 1 - 0, 1) = \max(0, 1, 1) = 1$
1, 1, 0	$\max(1 - 1, 1 - 1, 0) = \max(0, 0, 0) = 0$
1, 1, 1	$\max(1 - 1, 1 - 1, 1) = \max(0, 0, 1) = 1$

The two tables agree in all eight rows, so the rule and the clause are the same statement. Both are false in one row only, where both conditions hold and the conclusion does not.

Forward and backward chaining

Forward chaining is data driven. The engine repeatedly searches for rules whose conditions are all satisfied by known facts, adds their conclusions to the facts, and stops when no rule adds anything new. It suits monitoring and classification, where measurements arrive and the question is what follows from them. Backward chaining is goal driven. The engine starts from a hypothesis, finds rules that conclude it, and tries to establish their conditions, recursively, until it reaches known facts or must ask the user. It suits diagnosis, where the system checks a suspected fault and asks only the questions that matter for it [2, 9].

When several rules can fire at once, a conflict resolution strategy chooses among them, for example by rule priority, by specificity or by the recency of the facts involved. Large rule bases need efficient matching: The Rete algorithm compiles rule conditions into a network that remembers partial matches, so that each new fact is compared only with the conditions it can affect [10].

Animation: Forward chaining in a pump diagnosis rule base. Select the observed symptoms and step through the inference. Each step fires one rule whose conditions are satisfied and adds its conclusion to the facts. Run it on the [lecture page](#).

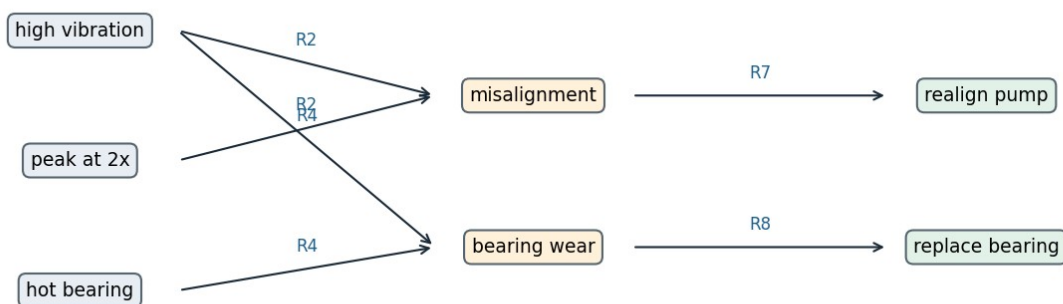


Figure 3.1. A small forward-chaining trace for pump diagnosis. Observations enter at the left, rules fire in sequence and conclusions lead to maintenance actions.

Check your understanding. A maintenance engineer suspects cavitation and wants the system to ask only the questions relevant to that suspicion. Which inference strategy fits?

- Forward chaining over all rules
- Backward chaining from the hypothesis

- C. Random rule selection
- D. Breadth-first search over sensor readings

Answer: B. Backward chaining starts from the goal and seeks only the evidence needed to confirm or reject it.

Formulas: Forward chaining as a growing set of facts

Let F_0 be the set of facts known at the start, the observations. Each rule r has a set of conditions C_r and a conclusion q_r . In the formulas below, $\subseteq$ reads “is contained in”, $\in$ reads “is an element of”, $\notin$ reads “is not an element of”, and $\cup$ joins two sets into one. At cycle t the conflict set holds every rule whose conditions are all known and whose conclusion is still new [9]:

$$CS_t = \{r : C_r \subseteq F_t \text{ and } q_r \notin F_t\}$$

Conflict resolution picks one rule r_t from the conflict set, here the rule with the lowest number, and the engine adds its conclusion to the facts:

$$F_{t+1} = F_t \cup \{q_{r_t}\}$$

The engine stops at the first cycle whose conflict set is empty. Each rule can add its conclusion only once, so a rule base with R rules stops after at most R firings. For rules in Horn form, forward chaining derives every fact that follows from the observations, and nothing else [9].

Matching is the expensive part. An engine that tests every condition of every rule in every cycle makes, over T cycles,

$$N_{\text{naive}} = T \times \sum_r |C_r|$$

tests, where $|C_r|$ is the number of conditions of rule r . The Rete algorithm instead compiles the conditions into a network that remembers partial matches, so that each new fact is tested only against the conditions that mention it [10].

Backward chaining works from a goal g . If g is already a fact, the goal is reached. Otherwise the engine takes each rule with $q_r = g$ in turn and tries to establish every condition in C_r as a new goal. A goal that no rule concludes is an observation, and the engine asks the user about it [1, 9].

Worked example: Forward chaining in the pump rule base

The pump rule base of the animation has ten rules:

Rule	Conditions	Conclusion
R1	high vibration AND peak at 1x running speed	unbalance
R2	high vibration AND peak at 2x running speed	misalignment
R3	crackling noise at the inlet AND low suction pressure	cavitation
R4	high bearing temperature AND high vibration	bearing wear
R5	leak at the shaft seal	seal failure
R6	cavitation	action: raise suction head or reduce flow
R7	unbalance	action: balance the impeller
R8	misalignment	action: realign pump and motor
R9	bearing wear	action: replace bearing
R10	seal failure	action: replace mechanical seal

The observations in F_0 are high vibration, peak at 2x running speed and high bearing temperature. Conflict resolution fires the rule with the lowest number.

Cycle 1 ($t=0$). The conflict set is $CS_0 = \{R2, R4\}$. Rule R2 has the lowest number and fires, so $F_1 = F_0 \cup \{\text{misalignment}\}$.

Cycle 2 ($t=1$). The conflict set is $CS_1 = \{R4, R8\}$. Rule R4 has the lowest number and fires, so $F_2 = F_1 \cup \{\text{bearing wear}\}$.

Cycle 3 ($t=2$). The conflict set is $CS_2 = \{R8, R9\}$. Rule R8 has the lowest number and fires, so $F_3 = F_2 \cup \{\text{action: realign pump and motor}\}$.

Cycle 4 ($t=3$). The conflict set is $CS_3 = \{R9\}$. Its only rule, R9, fires, so $F_4 = F_3 \cup \{\text{action: replace bearing}\}$.

Cycle 5 ($t=4$). No rule has all its conditions in F_4 and a new conclusion, so CS_4 is empty and the engine stops with 7 facts.

Counting the tests. The ten rules have $\sum_r |C_r| = 2+2+2+2+1+1+1+1+1+1 = 14$ conditions.

Testing all of them in each of the $T=5$ cycles costs $N_{\text{naive}} = 5 \times 14 = 70$ tests. An engine that tests each fact only against the conditions that mention it needs one test per mention: High vibration appears in 3 conditions, peak at 2x running speed, high bearing temperature, misalignment and bearing wear in one each, and the two actions in none, which adds up to 7 tests. The saving grows with the number of rules and facts [10].

Worked example: Backward chaining from a suspected fault

An engineer suspects cavitation, so the goal is $g = \text{cavitation}$.

Step 1. Cavitation is not a fact. Only rule R3 concludes it, so its conditions, crackling noise at the inlet and low suction pressure, become new goals.

Step 2. No rule concludes “crackling noise at the inlet”, so it is an observation. The engine asks: Is there a crackling noise at the inlet? The answer is yes, and the fact is added.

Step 3. “Low suction pressure” is also an observation. Before answering, the engineer asks why the question matters. The engine answers with the rule it is trying to establish: R3 concludes cavitation when crackling noise and low suction pressure both hold, and crackling noise is already known. The answer is yes.

Step 4. Both conditions of R3 hold, so R3 fires and cavitation is established. Asked how, the engine reports: Cavitation by R3, from crackling noise (answered yes) and low suction pressure (answered yes).

The engine asked two questions instead of all seven observations. Had the first answer been no, R3 would have failed at once and the second question would never have been asked. Part A of the [interactive lab](#) runs backward chaining for any goal of this rule base and answers why and how.

Explanation and uncertainty

An expert system can answer two questions that most learned models cannot answer directly. The question how is answered by the chain of rules that produced a conclusion. The question why, asked when the system requests information, is answered by the rule that it is trying to establish. In safety-relevant engineering work, such traceable reasoning supports review, audit and certification.

Real knowledge is uncertain. MYCIN attached a certainty factor between minus one and one to rules and facts [4]. A conclusion receives the certainty of its rule multiplied by the smallest certainty of its conditions, and two rules supporting the same positive conclusion combine as $CF = CF1 + CF2(1 - CF1)$, so

that evidence accumulates but never exceeds one. Certainty factors were a practical engineering compromise rather than a probability theory, and their difficulties motivated both probabilistic reasoning and fuzzy logic. Vague conditions such as “slightly high temperature” cannot be captured by crisp thresholds at all: A reading of 69.9 degrees and one of 70.1 degrees would fire different rules. Week 4 addresses this with fuzzy sets [11].

Check your understanding. Two independent rules support the conclusion “bearing wear” with certainty factors 0.6 and 0.5. What is the combined certainty factor?

- A. 0.5
- B. 0.8
- C. 1.1
- D. 0.3

Answer: B. $0.6 + 0.5 \times (1 - 0.6) = 0.8$. Supporting evidence accumulates but never exceeds one.

Formulas: Certainty factors

A certainty factor CF is a number between -1 and 1 attached to a fact or to a rule. The value 1 means certainly true, -1 certainly false and 0 unknown. In the original MYCIN model the certainty factor of a hypothesis is the difference between a measure of belief MB and a measure of disbelief MD, both between 0 and 1 [4]:

$$CF = MB - MD$$

The certainty of a premise with several conditions follows the formulas of logic: The weakest condition limits a conjunction, and the strongest condition carries a disjunction [2, 4]:

$$CF(A \wedge B) = \min(CF(A), CF(B)), CF(A \vee B) = \max(CF(A), CF(B))$$

A rule IF E THEN H carries its own certainty factor CF_R , given by the expert. When the premise is believed, the rule passes certainty to its conclusion:

$$CF(H) = CF_R \times CF(E)$$

MYCIN fired a rule only when the certainty of its premise exceeded 0.2 [4]. When two rules give certainties CF_1 and CF_2 for the same conclusion, these are combined by one of three formulas, chosen by their signs [2, 4]:

$$CF = CF_1 + CF_2(1 - CF_1) \text{ if both are at least } 0$$

$$CF = CF_1 + CF_2(1 + CF_1) \text{ if both are negative}$$

$$CF = \frac{CF_1 + CF_2}{1 - \min(|CF_1|, |CF_2|)} \text{ if the signs differ}$$

The vertical bars denote the absolute value, the number without its sign. The result does not depend on the order of the two rules and always stays between -1 and 1 . A third rule is combined with the result of the first two in the same way.

Worked example: Two rules that support the same conclusion

Two independent rules support the conclusion “bearing wear” with certainty factors $CF_1 = 0.6$ and $CF_2 = 0.5$, as in the knowledge check above. Both are positive, so the first formula applies:

$$CF = 0.6 + 0.5 \times (1 - 0.6) = 0.6 + 0.5 \times 0.4 = 0.6 + 0.2 = 0.8$$

In the opposite order the result is the same:

$$CF = 0.5 + 0.6 \times (1 - 0.5) = 0.5 + 0.6 \times 0.5 = 0.5 + 0.3 = 0.8$$

The second rule removes part of the remaining doubt, $1 - 0.6 = 0.4$, so the evidence accumulates but never exceeds one.

Worked example: Diagnosing the pump with certainty factors

The observations are believed with $CF(\text{high vibration}) = 0.9$, $CF(\text{peak at } 2x) = 0.8$ and $CF(\text{high bearing temperature}) = 0.6$. Three rules apply: R2 (high vibration AND peak at 2x THEN misalignment) with $CF_R = 0.7$, R5 (high bearing temperature AND peak at 2x THEN misalignment) with $CF_R = 0.4$, and R4 (high bearing temperature AND high vibration THEN bearing wear) with $CF_R = 0.6$.

Step 1. Premises. Each premise is a conjunction, so it takes the smaller certainty: $\min(0.9, 0.8) = 0.8$ for R2, $\min(0.6, 0.8) = 0.6$ for R5 and $\min(0.6, 0.9) = 0.6$ for R4. All three exceed 0.2, so all three rules fire.

Step 2. Conclusions of the single rules.

$$R2: 0.7 \times 0.8 = 0.56, R5: 0.4 \times 0.6 = 0.24, R4: 0.6 \times 0.6 = 0.36$$

Step 3. Combining R2 and R5 for misalignment. Both are positive:

$$CF(\text{misalignment}) = 0.56 + 0.24 \times (1 - 0.56) = 0.56 + 0.24 \times 0.44 = 0.56 + 0.1056 = 0.6656$$

Misalignment, with 0.6656, is ranked before bearing wear, with 0.36. Part B of the [interactive lab](#) shows the same numbers and updates them when the observations change.

Step 4. Evidence against the conclusion. A laser alignment check finds the shafts aligned, which the engineer believes with $CF(E) = 0.9$. A rule R11, IF the alignment check finds the shafts aligned THEN misalignment, carries $CF_R = -0.8$:

$$CF = -0.8 \times 0.9 = -0.72$$

The signs differ, so the third formula combines this with 0.6656:

$$CF = \frac{0.6656 + (-0.72)}{1 - \min(0.6656, 0.72)} = \frac{-0.0544}{1 - 0.6656} = \frac{-0.0544}{0.3344} = -0.1627$$

Step 5. A second piece of negative evidence. A thermal image shows no hot spot at the coupling, and a rule gives misalignment $CF = -0.3$ from it. Both values are now negative, so the second formula applies:

$$CF = -0.1627 + (-0.3) \times (1 + (-0.1627)) = -0.1627 + (-0.3) \times 0.8373$$

$$CF = -0.1627 + (-0.2512) = -0.4139$$

The evidence now speaks against misalignment, and the engineer looks for another cause of the peak at twice the running speed. Part C of the [interactive lab](#) repeats every formula of this section for any certainties.

Engineering standards as rule bases: The Unified Soil Classification System

Many engineering standards are rule bases written for people. The Unified Soil Classification System, based on Casagrande's airfield classification and standardised as ASTM D2487, assigns a group symbol to a soil from its grain-size distribution and its Atterberg limits [6, 7]. The rules proceed in stages. A soil is fine grained when at least half of it passes the No. 200 sieve (0.075 mm), and coarse grained otherwise. A coarse soil is a gravel when more than half of its coarse fraction is retained on the No. 4 sieve (4.75 mm), and a sand otherwise. A clean gravel with less than 5 percent fines is well graded (GW) when its coefficient of uniformity is at least 4 and its coefficient of curvature lies between 1 and 3, and poorly

graded (GP) otherwise; for sands the uniformity limit is 6. Gravels and sands with more than 12 percent fines are silty (GM, SM) or clayey (GC, SC) depending on the plasticity of the fines, and those with 5 to 12 percent fines receive dual symbols such as GW-GM.

Fine-grained soils are placed on Casagrande's plasticity chart, which plots the plasticity index PI against the liquid limit LL. The A-line, $PI = 0.73 (LL - 20)$, separates clays above it from silts below it. Soils with LL below 50 are lean clays (CL) when PI is above 7 and on or above the A-line, silts (ML) when PI is below 4 or the point lies below the A-line, and silty clays (CL-ML) in the band of PI from 4 to 7 above the A-line. Soils with LL of 50 or more are fat clays (CH) above the A-line and elastic silts (MH) below it [6]. The notebook turns these rules into a function, and the lab shows the reasoning path for any combination of test results.

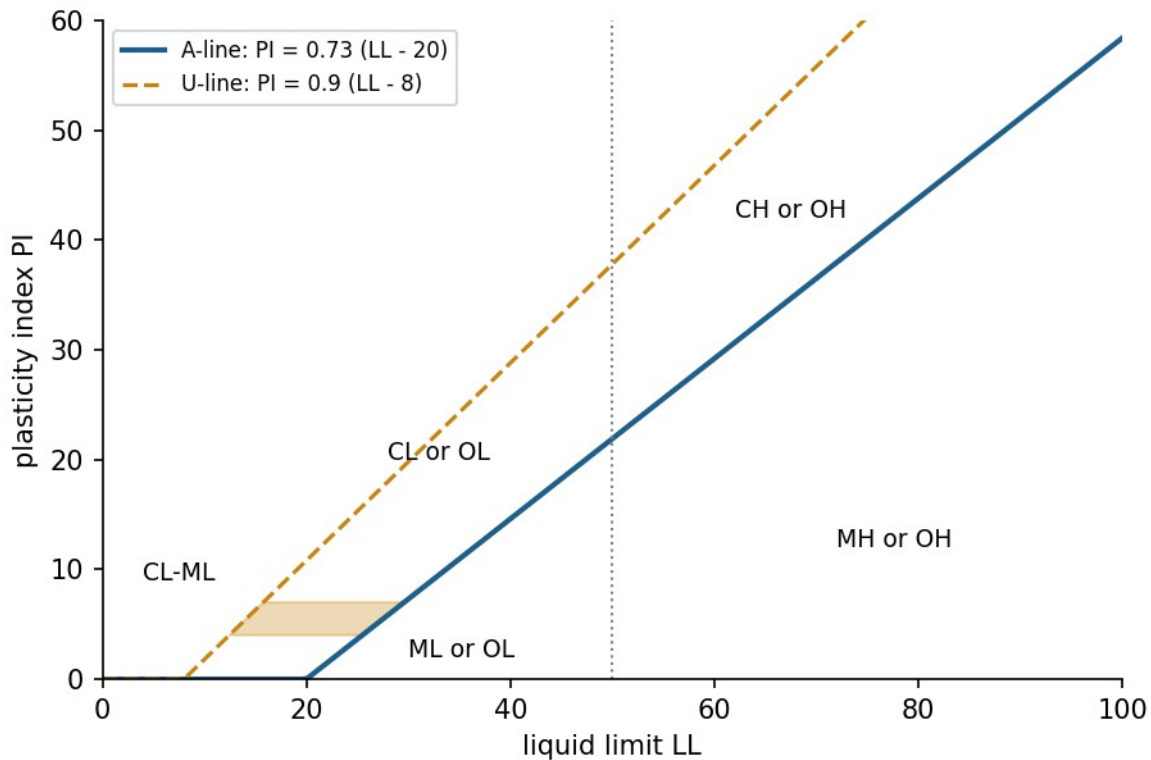


Figure 3.2. Casagrande's plasticity chart with the A-line and U-line used by the Unified Soil Classification System for fine-grained soils [6, 7].

Check your understanding. A fine-grained soil has $LL = 38$ and $PI = 18$. The A-line value at $LL = 38$ is $0.73 \times 18 = 13.1$. What is its group symbol?

- A. ML
- B. CL
- C. CH
- D. MH

Answer: B. LL is below 50, PI is above 7 and the point lies above the A-line, so the soil is a lean clay, CL.

Formulas: Grain size and plasticity

A sieve analysis shakes a dry sample of mass M through a stack of sieves, from the coarsest to the finest, and weighs the mass m_j retained on each sieve j . The percentage passing sieve i is the part of the sample that was retained neither on it nor on any coarser sieve:

$$P_i = 100 \times \left(1 - \frac{m_1 + m_2 + \dots + m_i}{M} \right)$$

The fines content is the percentage passing the No. 200 sieve, $F = P_{200}$, and the gravel content is the percentage retained on the No. 4 sieve, $G = 100 - P_4$ [6]. The coarse fraction is $100 - F$, and a coarse soil is a gravel when more than half of its coarse fraction is gravel:

$$\frac{G}{100 - F} > 0.5$$

The grain size D_x is the size than which x percent of the soil is finer. It is read from the grading curve, which plots the percentage passing against the sieve opening on a logarithmic scale. Between two neighbouring sieves with openings $D_a > D_b$ and percentages passing $P_a > P_b$, the curve is taken as a straight line on that scale, which gives, with $\log$ the logarithm to base 10,

$$\log D_x = \log D_b + \frac{x - P_b}{P_a - P_b} \times (\log D_a - \log D_b)$$

The coefficient of uniformity C_u and the coefficient of curvature C_c describe the shape of the curve [6]:

$$C_u = \frac{D_{60}}{D_{10}}, C_c = \frac{D_{30}^2}{D_{10} \times D_{60}}$$

A gravel is well graded when $C_u \geq 4$ and $1 \leq C_c \leq 3$, and a sand when $C_u \geq 6$ and $1 \leq C_c \leq 3$. The plasticity of the fines is measured by the liquid limit LL and the plastic limit PL, the water contents in percent at which the soil changes from plastic to liquid and from semi-solid to plastic behaviour. Their difference is the plasticity index, the range of water content over which the soil is plastic:

$$PI = LL - PL$$

On the plasticity chart, the A-line and the U-line are [6, 7]:

$$PI_A = 0.73 \times (LL - 20), PI_U = 0.9 \times (LL - 8)$$

Below $LL = 25.5$ the A-line runs horizontally at $PI = 4$. The U-line is an approximate upper limit for natural soils, so a point above it suggests that the test results should be checked [6].

Worked example: Classifying a sand from its sieve analysis

A dry sample of $M = 1000$ g gives the following masses on the sieves, and 60 g reach the pan:

Sieve	Opening (mm)	Retained m_j (g)	Cumulative (g)	Percent passing P_i
3/4 in	19	0	0	$100 \times (1 - 0/1000) = 100$
3/8 in	9.5	60	60	$100 \times (1 - 60/1000) = 94$
No. 4	4.75	90	150	$100 \times (1 - 150/1000) = 85$
No. 10	2	150	300	$100 \times (1 - 300/1000) = 70$
No. 20	0.85	170	470	$100 \times (1 - 470/1000) = 53$
No. 40	0.425	160	630	$100 \times (1 - 630/1000) = 37$
No. 60	0.25	130	760	$100 \times (1 - 760/1000) = 24$
No. 100	0.15	100	860	$100 \times (1 - 860/1000) = 14$
No. 200	0.075	80	940	$100 \times (1 - 940/1000) = 6$

Step 1. Fines and gravel. $F = P_{200} = 6$ percent and $G = 100 - P_4 = 100 - 85 = 15$ percent. Less than half of the soil passes the No. 200 sieve, so the soil is coarse grained. Its coarse fraction is $100 - 6 = 94$ percent, and

$$\frac{G}{100 - F} = \frac{15}{94} = 0.1596$$

is below 0.5, so the soil is a sand.

Step 2. The effective size D_{10} . It lies between the No. 100 sieve (0.15 mm, 14 percent passing) and the No. 200 sieve (0.075 mm, 6 percent passing). With $\log 0.075 = -1.12494$ and $\log 0.15 = -0.82391$:

$$\log D_{10} = -1.12494 + \frac{10 - 6}{14 - 6} \times (-0.82391 - (-1.12494))$$

$$\frac{10 - 6}{14 - 6} \times (-0.82391 - (-1.12494)) = \frac{4}{8} \times 0.30103 = 0.5 \times 0.30103 = 0.15052$$

$$\log D_{10} = -1.12494 + 0.15052 = -0.97442, D_{10} = 10^{-0.97442} = 0.1061 \text{ mm}$$

Step 3. The size D_{30} . It lies between the No. 40 sieve (0.425 mm, 37 percent passing) and the No. 60 sieve (0.25 mm, 24 percent passing). With $\log 0.25 = -0.60206$ and $\log 0.425 = -0.37161$:

$$\log D_{30} = -0.60206 + \frac{30 - 24}{37 - 24} \times (-0.37161 - (-0.60206))$$

$$\frac{30 - 24}{37 - 24} \times (-0.37161 - (-0.60206)) = \frac{6}{13} \times 0.23045 = 0.4615 \times 0.23045 = 0.10635$$

$$\log D_{30} = -0.60206 + 0.10635 = -0.49571, D_{30} = 10^{-0.49571} = 0.3194 \text{ mm}$$

Step 4. The size D_{60} . It lies between the No. 10 sieve (2 mm, 70 percent passing) and the No. 20 sieve (0.85 mm, 53 percent passing). With $\log 0.85 = -0.07058$ and $\log 2 = 0.30103$:

$$\log D_{60} = -0.07058 + \frac{60 - 53}{70 - 53} \times (0.30103 - (-0.07058))$$

$$\frac{60 - 53}{70 - 53} \times (0.30103 - (-0.07058)) = \frac{7}{17} \times 0.37161 = 0.4118 \times 0.37161 = 0.15303$$

$$\log D_{60} = -0.07058 + 0.15303 = 0.08245, D_{60} = 10^{0.08245} = 1.2091 \text{ mm}$$

Step 5. The two coefficients.

$$C_u = \frac{1.2091}{0.1061} = 11.4, C_c = \frac{0.3194^2}{0.1061 \times 1.2091} = \frac{0.10202}{0.12829} = 0.795$$

The uniformity coefficient passes the limit of 6 for sands, but C_c is below 1, so the sand is poorly graded, SP: The curve has too little material in the middle sizes.

Step 6. The fines. With 5 to 12 percent fines, the soil receives a dual symbol whose second part depends on the plasticity of the fines [6]. The fines have $LL = 30$ and $PL = 24$, so $PI = 30 - 24 = 6$. The A-line at this liquid limit is $0.73 \times (30 - 20) = 7.3$, and the point lies below it, so the fines are silty, ML, and the soil is SP-SM. Part D of the [interactive lab](#) computes the grading curve, the three sizes and the symbol from any sieve masses.

Worked example: Three fine-grained soils on the plasticity chart

Soil a. $PI = 38 - 20 = 18$. The A-line: $PI_A = 0.73 \times (38 - 20) = 0.73 \times 18 = 13.14$. The U-line: $PI_U = 0.9 \times (38 - 8) = 0.9 \times 30 = 27$, above the point, as expected for a natural soil. Since LL is below 50, PI is above 7 and the point lies above the A-line, the group symbol is CL.

Soil b. $PI = 62 - 38 = 24$. The A-line: $PI_A = 0.73 \times (62 - 20) = 0.73 \times 42 = 30.66$. The U-line: $PI_U = 0.9 \times (62 - 8) = 0.9 \times 54 = 48.6$, above the point, as expected for a natural soil. Since LL is at least 50 and the point lies below the A-line, the group symbol is MH.

Soil c. $PI = 22 - 16 = 6$. The A-line: LL is below 25.5, where the A-line runs horizontally at $PI_A = 4$ (the sloping formula would give $0.73 \times (22 - 20) = 1.46$). The U-line: $PI_U = 0.9 \times (22 - 8) = 0.9 \times 14 = 12.6$, above the point, as expected for a natural soil. Since LL is below 50 and PI lies between 4 and 7 on or above the A-line, the group symbol is CL-ML.

Soil	LL	PL	PI	A-line	U-line	Group symbol
a	38	20	18	13.14	27	CL
b	62	38	24	30.66	48.6	MH
c	22	16	6	4	12.6	CL-ML

Part E of the [interactive lab](#) places any pair of limits on the chart and prints the same reasoning path.

Strengths and limits

Knowledge-based systems are transparent, can be verified rule by rule, need no training data, and encode requirements that must hold exactly, such as the limits of a standard. They are brittle outside the cases their authors anticipated, they are expensive to acquire and maintain, and they cannot perceive raw signals or images. Modern practice therefore combines them with learning: A convolutional network may detect a crack, and a rule base derived from a code decides whether the crack width requires repair. Such hybrid systems return in later weeks, for example when a language model retrieves passages from standards in Week 13.

Python step 3: Dictionaries and functions

Dictionaries

A dictionary maps keys to values, like a small table with named entries. It is written with braces and colons, and values are read and written with square brackets and the key. The method `get` returns a default instead of an error when a key is missing, and `items` yields pairs of key and value for a loop. A laboratory result for a soil sample fits naturally into a dictionary.

```
sample = {"id": "BH2-3.5m", "fines_pct": 62.0, "LL": 48.0, "PL": 22.0}
print(sample["LL"], sample.get("organic", False))
sample["PI"] = sample["LL"] - sample["PL"]      # adds a new key
for key, value in sample.items():
    print(key, "=", value)
```

Output

```
48.0 False
id = BH2-3.5m
fines_pct = 62.0
LL = 48.0
PL = 22.0
PI = 26.0
```

Dictionaries can hold other containers, for example a list of samples, or a dictionary of samples keyed by borehole and depth. They are also the everyday format of records read from files and web services.

Functions

A function packages a calculation under a name so that it can be reused and tested. The keyword `def` starts the definition, the names in parentheses are parameters, and `return` sends the result back to the

caller. The string on the first line of the body, the docstring, documents what the function does. Variables created inside a function are local and disappear when it returns. The A-line of the plasticity chart and a simplified group decision for fine-grained soils make two short functions [6].

```
def a_line(ll):
    """Plasticity index on the A-line of the plasticity chart for
    liquid limit ll."""
    return 0.73 * (ll - 20)

def fine_soil_group(ll, pi):
    """Group symbol of an inorganic fine-grained soil (simplified)."""
    clay = pi > a_line(ll) and pi > 7
    if ll < 50:
        return "CL" if clay else "ML"
    return "CH" if clay else "MH"

print(round(a_line(48), 2))
print(fine_soil_group(48, 26), fine_soil_group(ll=62, pi=18))
```

Output

```
20.44
CL MH
```

The second call names its arguments, which makes it easier to read and independent of the order of the parameters. The full standard adds further cases, such as a dual symbol for low plasticity near the A-line and separate groups for organic soils; the simplified function shows the pattern, not the complete procedure. Parameters can also have default values, which a caller may omit.

```
def plasticity_check(pl, ll, margin=0.0):
    """True when the liquid limit exceeds the plastic limit by more
    than the margin."""
    return ll - pl > margin

print(plasticity_check(22, 48), plasticity_check(22, 48, margin=30))
```

Output

```
True False
```

Rules as data

Expert systems keep their knowledge as data, separate from the code that reasons with it. A list of dictionaries represents a small rule base for pump diagnosis, and a set holds the observed facts. The built-in function `all` returns `True` when every item of a sequence is true, so `all(c in facts for c in rule["if"])` asks whether every condition of a rule is among the facts. Repeating the pass until no rule adds a new fact is forward chaining, the inference method of this week.

```
rules = [
    {"name": "R1", "if": ["low suction pressure", "crackling noise"],
    "then": "cavitation"},
    {"name": "R2", "if": ["high vibration", "peak at running speed"],
    "then": "imbalance"},
    {"name": "R3", "if": ["cavitation"], "then": "check suction line
and NPSH margin"},
]
```

```

facts = {"low suction pressure", "crackling noise", "high vibration"}

changed = True
while changed:
    changed = False
    for rule in rules:
        if all(c in facts for c in rule["if"]) and rule["then"] not in
facts:
            facts.add(rule["then"])
            print(rule["name"], "fired:", rule["then"])
            changed = True

```

Output

R1 fired: cavitation
R3 fired: check suction line and NPSH margin

Rule R2 does not fire, because one of its conditions is missing. Adding knowledge requires no change to the reasoning code, only a new dictionary in the list, which is the main practical advantage of keeping rules as data.

Check your understanding. After `def area(b, h=0.5): return b * h`, what is the value of `area(2) + area(2, h=1)`?

- A. 3.0
- B. 2.0
- C. 4.0
- D. An error, because h is missing in the first call

Answer: A. The first call uses the default $h = 0.5$ and returns 1.0, and the second passes $h = 1$ and returns 2.

Discipline challenges

Standards and diagnostic procedures exist in every department. Pick the row closest to your field and write the rules; start with five and test them with the engine of the notebook.

Department	Challenge
Civil Engineering	Full USCS classification including dual symbols, and a check of whether a soil is suitable as a road subgrade.
Geological Engineering	Rock mass description rules, for example classes from joint spacing and weathering grade.
Mechanical Engineering	Pump or fan fault diagnosis from vibration spectrum features, extending the rule base of the lecture.
Electrical and Electronics Engineering	Transformer condition rules from dissolved-gas ratios, written as a decision table.
Environmental Engineering	Discharge compliance checks of treated wastewater against permit limits.
Food Engineering	Hazard analysis rules that assign critical control points in a production line.
Chemistry	Rules that assign a functional group from infrared absorption bands.
Mining Engineering	Ventilation alarm rules that combine methane concentration, airflow and trends.

Department	Challenge
Computer Engineering	Firewall or access-control policy evaluation with rule conflicts and priorities.
Industrial Engineering	Dispatching rules for a job shop, compared on a small scenario.
Textile Engineering	Defect-cause rules that link fabric faults to loom settings.

Weekly task

Choose a classification procedure, a design check or a diagnostic guideline from your department and implement at least eight rules in the notebook's rule engine. Test it on five cases, show the forward-chaining trace for one of them, and write about 400 words on what the rules cover, where they are brittle and how a domain expert could review them. Cite at least three works from this week's references [2, 4, 10].

Research and report assignment (optional)

Expert systems in engineering practice. Review the history and present use of rule-based systems in one engineering domain, such as process control alarms, building code checking, geotechnical classification or maintenance diagnostics. Discuss knowledge acquisition, verification and the combination of rules with machine learning [1, 2, 5].

Both assignments are optional. During an active semester, the weekly task can be sent together with the learning log exported from the interactive lab, and the research report on its own, to utkukose@sdu.edu.tr or utkukose@gmail.com. They carry no separate weight, but the instructor may take them into account as a discretionary adjustment of the midterm component of the grade. The assessment section of the syllabus gives the details, and research reports follow the report template of the course repository.

References

- [1] Hayes-Roth, F., Waterman, D. A., & Lenat, D. B. (Eds.) (1983). *Building Expert Systems*. Addison-Wesley.
- [2] Giarratano, J. C., & Riley, G. D. (2005). *Expert Systems: Principles and Programming* (4th ed.). Thomson Course Technology.
- [3] Lindsay, R. K., Buchanan, B. G., Feigenbaum, E. A., & Lederberg, J. (1993). DENDRAL: A case study of the first expert system for scientific hypothesis formation. *Artificial Intelligence*, 61(2), 209-261. [https://doi.org/10.1016/0004-3702\(93\)90068-M](https://doi.org/10.1016/0004-3702(93)90068-M)
- [4] Buchanan, B. G., & Shortliffe, E. H. (Eds.) (1984). *Rule-Based Expert Systems: The MYCIN Experiments of the Stanford Heuristic Programming Project*. Addison-Wesley.
- [5] McDermott, J. (1982). R1: A rule-based configurer of computer systems. *Artificial Intelligence*, 19(1), 39-88. [https://doi.org/10.1016/0004-3702\(82\)90021-2](https://doi.org/10.1016/0004-3702(82)90021-2)
- [6] ASTM International (2017). *ASTM D2487-17: Standard Practice for Classification of Soils for Engineering Purposes (Unified Soil Classification System)*. ASTM International.
- [7] Casagrande, A. (1948). Classification and identification of soils. *Transactions of the American Society of Civil Engineers*, 113, 901-930.
- [8] Python Software Foundation (2026). *The Python Tutorial (Python 3 documentation)*. <https://docs.python.org/3/tutorial/>
- [9] Russell, S., & Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson.

- [10] Forgy, C. L. (1982). Rete: A fast algorithm for the many pattern/many object pattern match problem. *Artificial Intelligence*, 19(1), 17-37. [https://doi.org/10.1016/0004-3702\(82\)90020-0](https://doi.org/10.1016/0004-3702(82)90020-0)
- [11] Zadeh, L. A. (1965). Fuzzy sets. *Information and Control*, 8(3), 338-353. [https://doi.org/10.1016/S0019-9958\(65\)90241-X](https://doi.org/10.1016/S0019-9958(65)90241-X)